CS201 Homework 2

Name: Yakup Emre

Surname: Çelebi

Section: 2

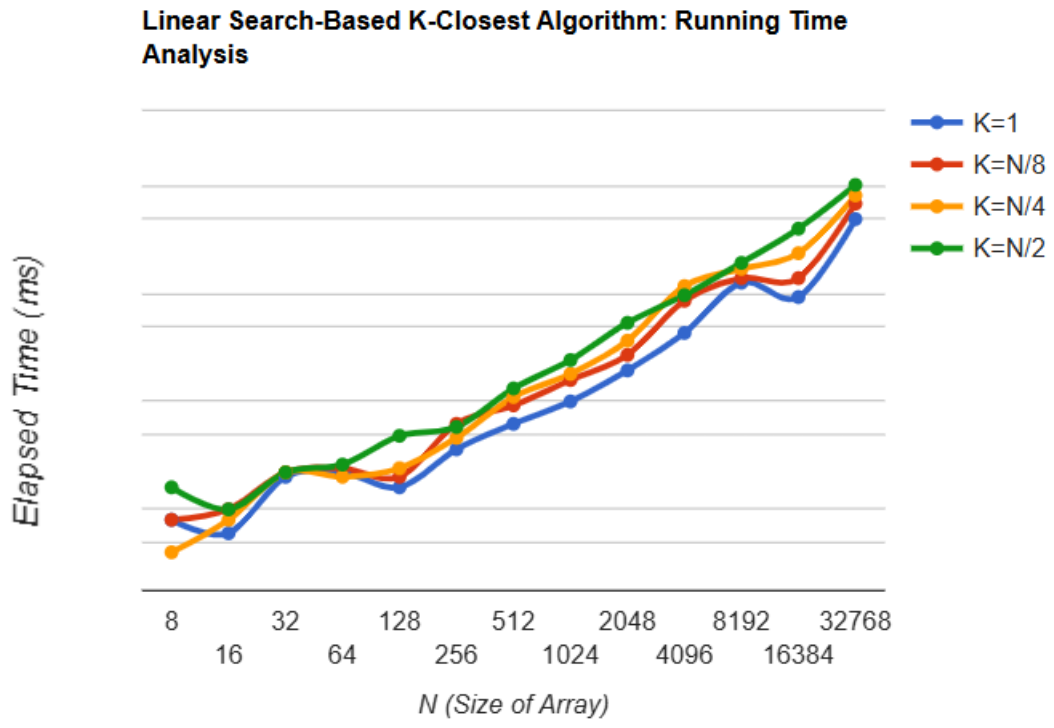
Student ID: 22302267

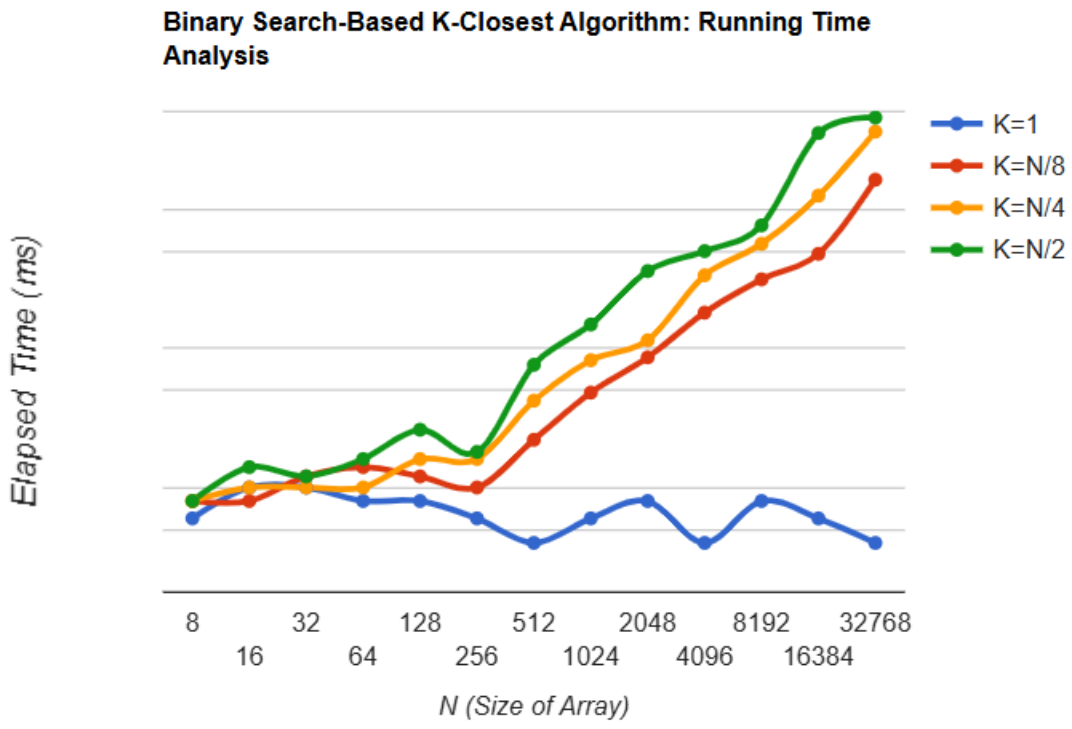
Date: 19.11.2025

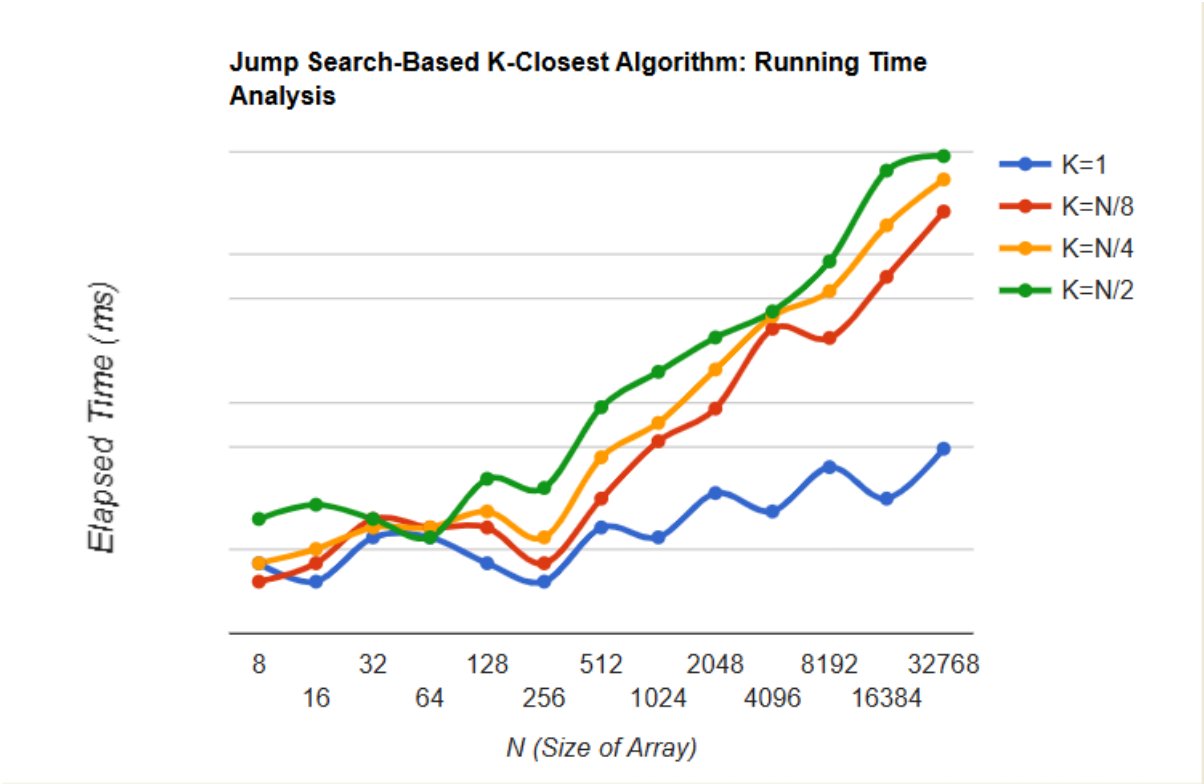
Running Time Results (in milliseconds) for Adapted Search Algorithms

N	Linear Search (ms)				Binary Search (ms)				Jump Search (ms)			
K	K=1	K=N/8	K=N/4	K=N/2	K=1	K=N/8	K=N/4	K=N/2	K=1	K=N/8	K=N/4	K=N/2
2^3	0.000080	0.000080	0.000040	0.000160	0.000060	0.000080	0.000080	0.000080	0.000080	0.000060	0.000080	0.000160
2^4	0.000060	0.000100	0.000080	0.000100	0.000100	0.000080	0.000100	0.000140	0.000060	0.000080	0.000100	0.000200
2^5	0.000200	0.000220	0.000220	0.000220	0.000100	0.000120	0.000100	0.000120	0.000120	0.000160	0.000140	0.000160
2^6	0.000220	0.000240	0.000200	0.000260	0.000080	0.000140	0.000100	0.000160	0.000120	0.000140	0.000140	0.000120
2^7	0.000160	0.000200	0.000240	0.000480	0.000080	0.000120	0.000160	0.000260	0.000080	0.000140	0.000180	0.000300
2^8	0.000360	0.000620	0.000460	0.000580	0.000060	0.000100	0.000160	0.000180	0.000060	0.000080	0.000120	0.000260
2^9	0.000620	0.000920	0.001100	0.001320	0.000040	0.000220	0.000420	0.000760	0.000140	0.000220	0.000420	0.000920
2^{10}	0.001000	0.001580	0.001800	0.002420	0.000060	0.000480	0.000820	0.001480	0.000120	0.000540	0.000720	0.001600
2^{11}	0.001940	0.002700	0.003660	0.005360	0.000080	0.000860	0.001140	0.003580	0.000240	0.000900	0.001660	0.002740
2^{12}	0.004320	0.008580	0.011740	0.009660	0.000040	0.001800	0.003340	0.004980	0.000180	0.003140	0.003820	0.004120
2^{13}	0.012620	0.013880	0.016940	0.019260	0.000080	0.003120	0.005620	0.007620	0.000360	0.002720	0.005660	0.009020
2^{14}	0.009280	0.013920	0.023680	0.039980	0.000060	0.004760	0.012460	0.035040	0.000220	0.007080	0.015840	0.037320
2^{15}	0.049040	0.068260	0.081300	0.101940	0.000040	0.016220	0.035900	0.045260	0.000480	0.019680	0.032500	0.046820

Plots


Linear Search-Based K-Closest Algorithm: Running Time Analysis
K=1
K=N/8
K=N/4
K=N/2
Elapsed Time (ms)
8
16
32
64
128
256
512
1024
2048
4096
8192
16384
32768
N (Size of Array)


Binary Search-Based K-Closest Algorithm: Running Time Analysis
K=1
K=N/8
K=N/4
K=N/2
Elapsed Time (ms)
8
16
32
64
128
256
512
1024
2048
4096
8192
16384
32768
N (Size of Array)



Analysis of Results and Discussion

Computer Specifications:

- **Processor (CPU):** 12th Gen Intel(R) Core(TM) i5-12500H
  - **Core Details:** 12 Physical Cores (P/E cores), 16 Logical Processors (Threads).
  - **Base Clock:** 2.50 GHz.
- **System Memory (RAM):** 16.0 GB (DDR5).
- **Operating System (OS):** Microsoft Windows 11 Home.

Algorithm Suitability for Unsorted Data:

Algorithm 1 which is linear search works when the data is unsorted since it iterates all over the elements in the given data. However, algorithm 2 and 3 which are binary search and jump search do not work when the data is unsorted.

Possible Scenarios and the Time Complexities for the Algorithms:

For linear search, the best case is finding it in the first comparison and the time complexity is O(1). The worst case is iterating all over the elements, and the time complexity for the worst case is O(n). Also, the average case is making n/2 comparisons and the time complexity for the average case is O(n).

For binary search, best case is finding it in the first comparison and the time complexity is O(1) for the best case. The worst case is finding it in the last comparison, and the time complexity for the worst case is O(log n). Average case is also O(log n).

For jump search, the best case is finding it in the first comparison and the time complexity is O(1). The worst case is going through the end of the block and going through the end of the found block, the time complexity for it is O(N/m + m) where m is the block size and n is the number of elements. In optimal block size m= sqrt(n), time complexity is O(sqrt(n)).

In adapted algorithms, we can split algorithm into two parts: finding the position of key value, and finding the closest k values to the key value. These cases do not change compared to first part of the algorithm. Time complexities are the same, however for finding the closest k value, the time complexity is O(k).

Linear Search (Algorithm 1):

	Searching Algorithm	Adapted Algorithm
Best Case	O(1)	O(1+K)
Average Case	O(n)	O(n + K)
Worst Case	O(n)	O(n + K)

Binary Search (Algorithm 2):

	Searching Algorithm	Adapted Algorithm

Best Case	O(1)	O(1+K)
Average Case	O(log n)	O(K + log n)
Worst Case	O(log n)	O(K + log n)

Jump Search (Algorithm 3):

	Searching Algorithm	Adapted Algorithm
Best Case	O(1)	O(1+K)
Average Case	O(sqrt(n))	O(K + sqrt(n))
Worst Case	O(sqrt(n))	O(K + sqrt(n))

Worst, Average, and Best Cases for the Algorithms based on the Plots

In all three algorithms, the best case occurs when k=1 which the elapsed time is the least. The expected situation is also the best case when n is minimum, that is, 8. However, based on the algorithm with the linear search and jump search plot, base case is when N = 16 since the elapsed time is minimum. Also, interestingly, the elapsed time was observed to be at least when n = 32768. Theoretically, this would be expected when n = 8. Also, the worst cases for 3 algorithms happen when the k = n/2 and n is maximum based on the plots. We can also call the average case when k=n/4 or k=n/8.

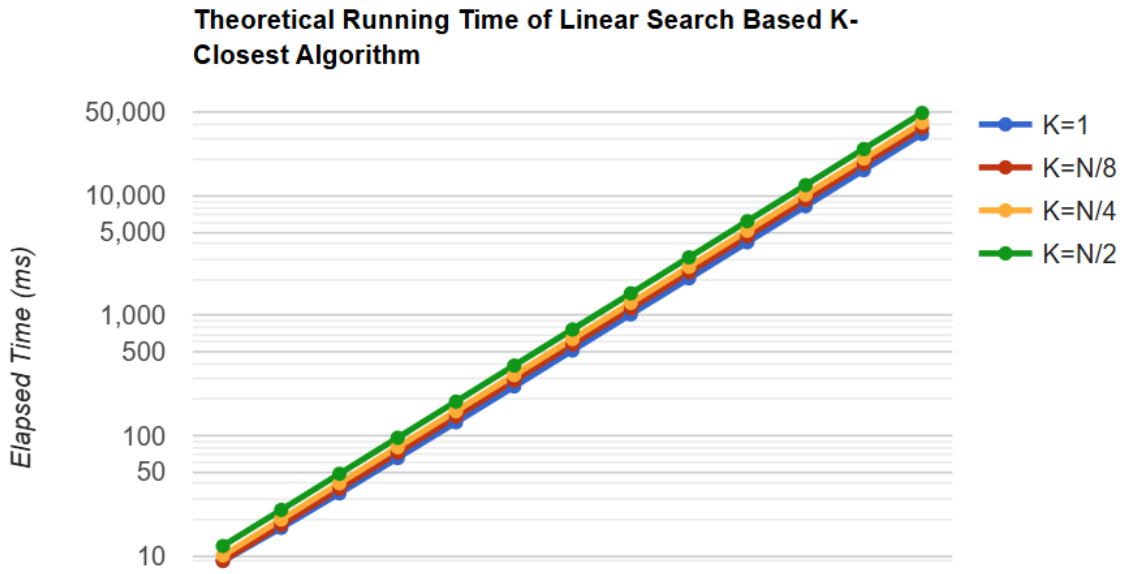
Therefore, although there are some minor fluctuations, the plot created according to the obtained data is in harmony with the theoretically expected plot in terms of general structure.

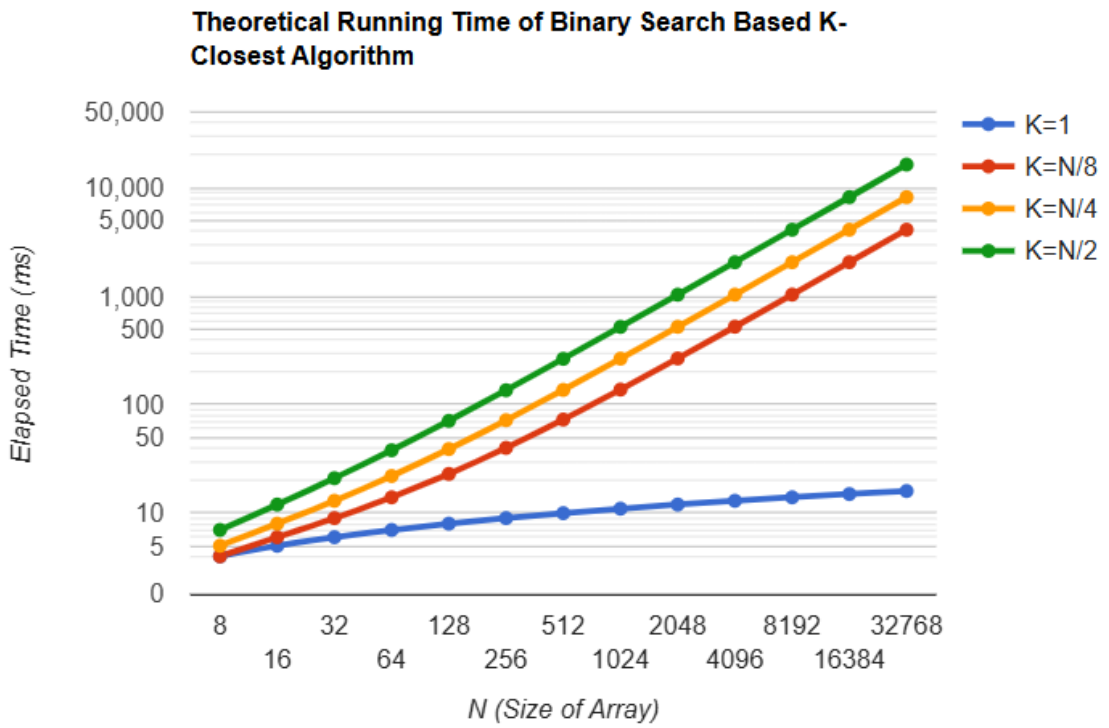
Why are There Differences?

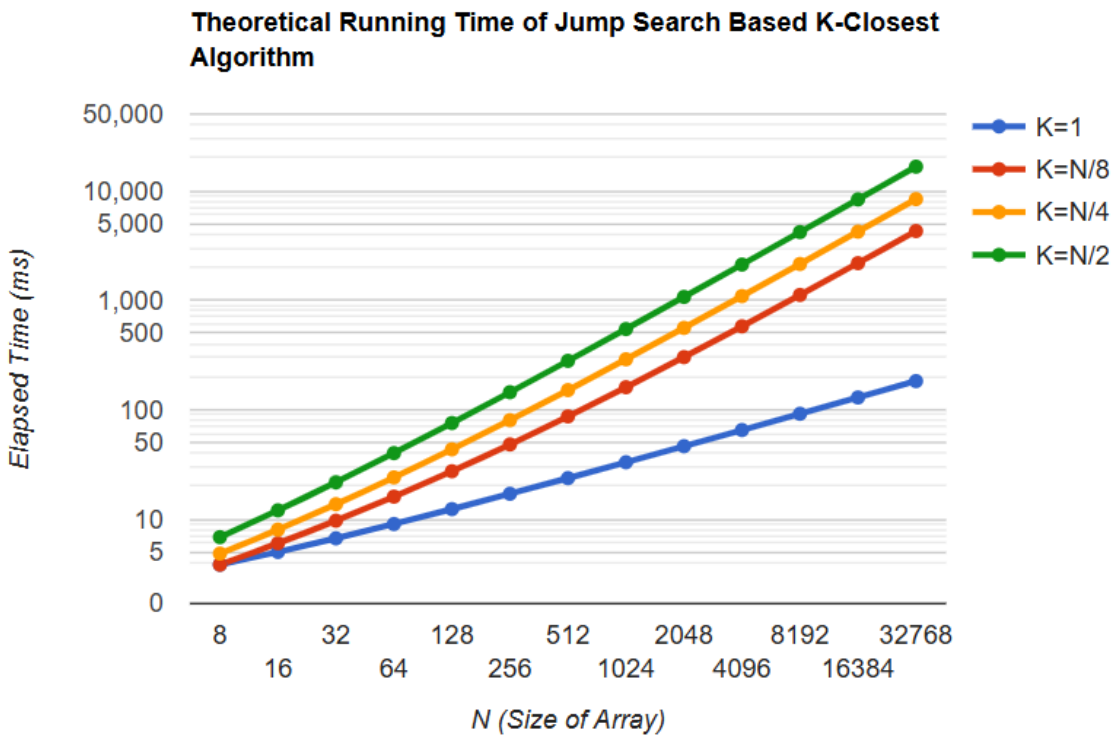
In binary search, the minimum time elapsed in the case of k=1 is when N is maximum. Since binary search is a very fast algorithm, a small change in the state of the computer such as increased load on the operating system can have more effect than the value of N. Also, changing the number of applications running in the background can cause fluctuations in the real-world plots.

Moreover, the random generation of the data also affects the speed of the algorithm. For example, if searched value is found by chance in the first comparisons, the algorithm's running time may be short even if N is very large.

Plots of Theoretical Running Times of Searching Algorithms by N and K


Theoretical Running Time of Linear Search Based K-Closest Algorithm
Elapsed Time (ms)
50,000
10,000
5,000
1,000
500
100
50
10
K=1
K=N/8
K=N/4
K=N/2


Theoretical Running Time of Binary Search Based K-Closest Algorithm
Elapsed Time (ms)
50,000
10,000
5,000
1,000
500
100
50
10
5
0
8
16
32
64
128
256
512
1024
2048
4096
8192
16384
32768
N (Size of Array)
K=1
K=N/8
K=N/4
K=N/2



Comparisons of Test Result Running Times Plots and Theoretical Running Times Plots

Comparison of Theoretical and Observed Running Times (Linear Search):

The comparison between the theoretical plot and the observed data confirms that the implemented algorithm behaves as expected for a linear complexity function.

Consistencies:

The plot of algorithm with the observed data displays almost straight line that is similar to the plot of theoretical running times of the algorithm that has O(N+K) time complexity growth as a linear function.

In both plots, the curves maintain the exact same vertical hierarchy: K=N/2 (Green) is always the highest, and K=1 (Blue) is always the lowest.

Inconsistencies:

The theoretical plot is perfectly smooth, as it assumes constant time for every operation such as comparison. On the other hand, the plot that is constructed with observed data results is jagged and noisy. The observed plot contains several minor dips at some points such as N = 16 and N = 16384.

Therefore, the theoretical model correctly predicts the rate of growth but the experimental data, while confirming the rate, adds real-world noise.

Comparison of Theoretical and Observed Running Times (Binary Search):

Consistencies:

In both graphs, the K=1 curve (Blue) is the most clearly horizontal, completely separate from the other three curves. This proves that N dominates logN as N grows.

The vertical order of the curves is preserved in both graphs.

Inconsistencies:

The theoretical graph is perfectly smooth, while the experimental graph shows peaks and valleys and fluctuations, especially in the range N < 512. This noise happens because operations like comparisons are the unit of nanosecond and the measured times are easily affected by external factors such as operating system or memory load, CPU usage percentage.

Therefore, the experimental plot shows the transition from log N to N successfully but it also shows the noises.

Comparison of Theoretical and Observed Running Times (Jump Search):

Consistencies:

In both graphs, the K=1 curve (Blue) is the most clearly horizontal, completely separate from the other three curves. This proves that the complexity of that curve is sqrt(n) and N dominates sqrt(n) as n grows.

The vertical order of all curves is preserved: curve K=N/2 is always at the top.

The curve K=N/2 approaches a straight line as N grows, indicating that the growth rate is shifting from O(sqrt(N)) to O(N).

Inconsistencies:

The theoretical graph is perfectly smooth, while the experimental graph shows peaks and valleys and fluctuations, especially in the range $N < 8192$.

Also, the $k=1$ curve fluctuates constantly, unlike the theoretical curve.